The Log Is the Agent: Engineering Inspectable Agent Runs

Yohei Nakajima

Preparation draft, November 2, 2026

Editorial review copy. Author metadata, declarations and final submission approval remain pending.

Abstract

Long-running agents need inspectable histories as well as useful outputs. ActiveGraph combines an append-only event history, a projected domain graph and reactive behaviors to expose how recorded goals, model responses and artifacts connect. We describe practical boundaries for state reconstruction, execution verification and branching, using a synthetic diligence fixture on a pinned implementation. Rebuilding a graph from accepted events differs from re-executing behaviors; a strict replay check does not establish arbitrary payload equality. Likewise, reusing recorded responses avoids some new model calls without making a changed branch a causal experiment. A bounded offline check reproduces the fixture's projected state, identifies timing differences in raw logs and checks branch isolation. Comparison with Temporal and LangGraph places these mechanisms in their established context. The resulting guidance concerns capture boundaries, reproducibility checks and safe intervention points, rather than improved task accuracy or demonstrated self-improvement.

Three actionable insights

- Specify whether a replay reconstructs recorded state, checks selected execution invariants or executes new work; verify the required equality directly.
- Treat a branch as a versioned intervention with explicit cache and side-effect boundaries, and retain the parent as a comparison artifact.
- Store enough source, code and environment provenance to audit a captured result, while separating traceability from correctness and permission to act.

Start with the question an engineer must answer

An agent produces a diligence memo saying a company has a concentration risk. An engineer needs to find the question that prompted the research, the source document, the model request and the response from which the claim was extracted. A final memo and a chat summary may omit these connections. Keeping a transcript helps, but an application must still decide which records govern its state and how to rebuild that state after a restart.

ActiveGraph makes that decision explicit: accepted events define a run's history, and a graph of typed objects and relations is projected from it. Behaviors subscribe to event types and graph

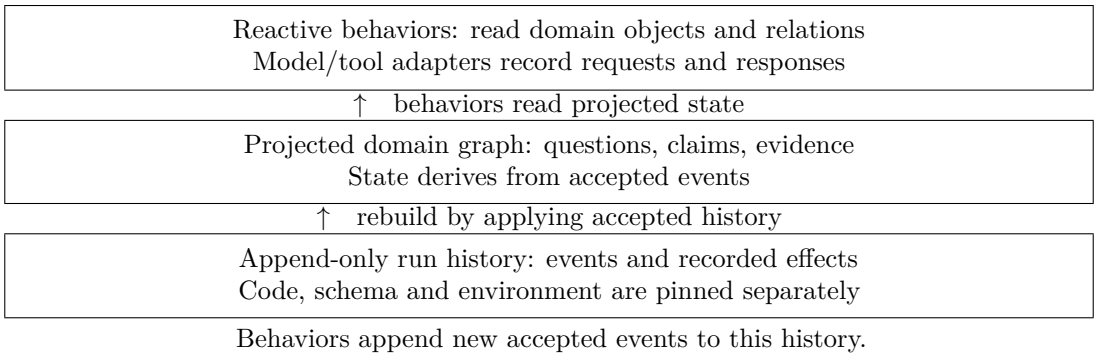


Figure 1: Accepted events define the captured run. Projection rebuilds domain state; behaviors produce new recorded work. External facts, unlogged I/O and code not retained with the run remain outside the capture boundary. Event emission does not authorize external actions.

patterns, then emit more events. The graph represents domain entities such as companies, questions, claims and supporting evidence. It is distinct from a diagram of workflow control flow. Figure 1 shows the design and the capture boundary that constrains its claims.

This article develops the engineering argument of the earlier architecture preprint [1], with corrected, implementation-specific guarantees. The baseline is source revision `7070112` of ActiveGraph [2], whose package metadata reads 1.13.0. That identity is not a claim that this commit is the published PyPI release. The contribution here is a bounded account of how authoritative history, a domain graph and recorded effects work together, including where the combination fails to provide a guarantee. Event sourcing, shared-state coordination and durable replay have substantial prior art.

Choose history authority before choosing storage

Event sourcing reconstructs state from an ordered record of changes; Fowler also describes reconstruction, alternative timelines and the need to handle external queries and updates explicitly [3]. ActiveGraph applies this pattern to agent applications. An object creation, a relation and a patch enter through the event surface. Runtime provenance links captured changes to their actor and causing event. A materialized graph makes questions about relationships convenient: which evidence supports a claim, which question remains unanswered, or which artifact depends on a disputed claim?

A behavior combines a subscription with code. When matching events arrive, the scheduler supplies the triggering event, graph access and execution context. Ordinary functions, classes and model-backed routines can participate; logic can also attach to typed relations. Coordination therefore moves into subscriptions and shared domain state. It does not disappear. Developers still choose schemas, triggers, policies and budgets, and must consider cycles and unintended cascades.

This resembles the older blackboard pattern, in which knowledge sources coordinate through a shared structure [4]. Using a language model as a knowledge source does not eliminate control design, unreliable inference or evaluation. The practical design choice is to retain the changes and their relationships so engineers can inspect them, rather than to claim a new general theory of coordination.

The authority boundary is application-specific. An application may use an ActiveGraph log as

its primary execution record, while another treats accepted Git records as canonical and derives a runtime graph from them. These are different deployments of the same machinery. A runtime trace must not silently become the authority for an external research conclusion, institutional decision or approval.

Give three different operations different contracts

State reconstruction applies recorded events to an empty projection. In the inspected implementation, `Runtime.load` performs this projection without firing the recorded behaviors. Its result depends on the event bytes and compatible projector/schema code. It does not ask a model to reproduce its old answer. Loading also prepares a runtime that may later continue unfinished work; that later execution must not be confused with the initial reconstruction.

Execution verification re-fires behavior code against recorded inputs and checks selected invariants. ActiveGraph's strict mode compares the event-type stream and length, with additional recorded model-request hash checks. It is not a byte comparison of every event payload. The accompanying boundary check changes a behavior's object text while preserving the event types and shows that strict replay accepts it. Adding an extra object event is rejected. A passing check therefore supports the invariants actually compared, not arbitrary behavior determinism, full state equality or the absence of all wall-clock and mutable-global dependencies.

New execution runs behaviors after loading or branching. It may call a model or tool, incur cost or affect another system. Recorded responses can be reused under explicit cache settings when request identities match; a changed request can require a fresh response. A no-new-calls claim must name the operation and cache policy, with a guard that fails if external access is attempted.

Separating these contracts changes debugging practice. If the goal is to verify a published projection, compare the projection's specified fields directly. If it is to detect a changed prompt, inspect the request hash and relevant failure. If it is to rerun an experiment, identify its fixed inputs, code, random streams, response records and scoring procedure. A single green "replay" label cannot substitute for these different checks.

The log also does not contain every dependency. Python behavior implementations, schemas, adapter versions and configuration need versioned references. A recorded model identifier describes the recorded request; it does not reproduce a provider's internal model state. Missing source documents or uncaptured direct I/O can break the path from an artifact back to inspectable evidence even when the graph rebuilds successfully.

Understand the nearest alternatives

Temporal is the closest first comparison for durable history and recorded effects. Its workflow replay checks commands against event history; completed activity results are recorded, and workflow logic has determinism constraints [5]. Its reset operation copies history to a valid workflow-task boundary and starts a new execution [6]. These mechanisms already provide durable replay and reuse of historical effects. ActiveGraph's event-sourced history is not novel simply because it contains model or tool results.

LangGraph offers checkpoint-based time travel and branching with modified state [7]. Work before the selected checkpoint is retained. Nodes after that point execute again, including model and API

Question	Temporal	LangGraph	ActiveGraph baseline
Saved representation	Workflow event history	Checkpointed graph state	Event history and projected domain graph
Replay meaning	Check workflow commands against history	Resume and re-execute nodes after a checkpoint	Reconstruct state; optional selected-invariant verification
Branch boundary	Reset at valid workflow-task boundary	Fork from saved checkpoint	Copy an inclusive valid event prefix; SQLite runtime fork
Review question	Which activities already completed?	Which suffix calls execute again?	Which events, responses and code define the branch?

Table 1: Different replay and branching contracts. Consult the named APIs and versions before treating two features with the same name as equivalent.

calls, which may return different results. It would be inaccurate to say such a fork must repay the saved prefix. Re-executing the suffix is a different contract from projecting recorded responses or validating behavior re-execution.

Table 1 summarizes documented distinctions rather than ranking systems. These observations are current as of November 2, 2026; they do not establish historical priority or tested implementation parity. ActiveGraph's reason to exist must be the usefulness of its reactive domain graph and captured lineage for a particular application, not an unsupported assertion that other frameworks cannot replay or branch.

Use a fixture to expose the mechanism and its limits

The bundled diligence pack processes three synthetic companies. Behaviors generate questions, consult bundled documents, extract claims and evidence, identify contradictions and risks, and produce memos. The provider is `RecordedDiligenceProvider`; it returns fixture responses. The model name in those response records is not evidence of a new provider request during this check. Fixture cost fields are not an invoice, and fixture/model latency fields must not be interpreted as measured end-to-end application performance.

The companion verification script constructs the same fixture configuration twice in a disposable directory, with a frozen event clock and fixed run identifier. A socket guard rejects attempted network connections or listening sockets. It serializes full events and projected objects/relations separately, reloads the saved run, and forks at a valid completed cutoff. The fork receives one additional annotation, after which the script checks that the parent's events remain unchanged. This demonstrates isolation at that cutoff, not every possible mid-execution intervention or crash-recovery case.

The checked fixture produced 671 events, 93 objects and 76 relations, with 103 fixture-served model-request events and 48 tool-request events. The receipt records the exact event, object, relation and request counts, the two raw event hashes, projection hash and field-level differences. Raw event logs differ because tool responses include measured local latency. Removing only `payload.latency_seconds` from `tool.responded` records defines a separate, explicitly normal-

ized comparison. Equality under that normalization must not be relabeled raw byte equality. The main engineering lesson is to choose the comparison fields before interpreting a successful check.

For a memo claim, the graph can connect the claim to its question, document and evidence; event provenance links the creation to the producing behavior and causing event. This makes a captured derivation inspectable. It does not establish that the synthetic claim is true, that the source was reliable or that the same procedure performs well on actual investment work. No task-accuracy comparison, provider-attested capture, independent replication or self-improvement result is demonstrated here.

Make branches explicit interventions

A runtime fork copies the parent's recorded prefix through an inclusive cutoff into a separate run and projects it. Copying and projection require local work and storage, even when they require no new model calls. The inspected runtime fork supports SQLite; it rejects cutoffs inside atomic promotion blocks. It does not itself invoke the strict execution verifier. Accordingly, a branch is not automatically an audit of how the prefix was originally produced.

With response-cache flags enabled, the fork can draw on recorded responses from the parent's full history, not only responses before the cutoff. This can be useful for reusing expensive work, but it changes the interpretation of an intervention. A purported fresh alternative must state whether later parent responses are available and whether matching requests receive historical answers. Otherwise the comparison can accidentally import information the alternative would not have had.

Use a cutoff whose invariants are understood. Retain the parent, record the changed configuration and declare which effects may execute again. Structural differences identify changed objects and relations; judging whether they constitute an improvement requires an application-specific evaluator. A different memo is not automatically a better memo, and a branch of software execution is not an identified causal experiment on the external world.

Real tools need a separate action boundary. Recording a write's response cannot undo a payment, email or database change. New execution can repeat effects unless the application controls them with idempotency keys, authorization, dry-run adapters or other appropriate safeguards. Logs should distinguish proposals, approvals and confirmed actions; replayability is not permission to perform them.

Budget for the responsibilities the log creates

The architecture adds operational responsibilities. Behavior cascades can loop or expand unexpectedly, so event/model/tool budgets and termination conditions matter. Response records occupy storage and may contain sensitive prompts or source content. Access control, retention, redaction and deletion obligations require an application policy; append-only storage is not an exemption from those obligations.

Schemas and projector code evolve. Keep compatible code and migration information with the run, and verify any snapshot/compaction boundary before claiming that an archived prefix can be reconstructed. The selected runtime has later snapshot and promotion features that the May preprint did not describe; this fixture does not evaluate their retention or concurrency properties. A single ordered log also does not establish safety for multiple independent writers.

An engineer choosing this design should ask whether typed relationships and traceable changes answer a concrete operational question. A linear workflow with adequate checkpoints may be simpler when the process is fixed and graph-level queries provide little value. Conversely, a question–claim–evidence graph can make the application boundary easier to inspect when several behaviors contribute different kinds of artifacts. That is an architectural fit judgment, not a measured superiority result.

Lessons for trustworthy software

Begin with a small capture contract: the accepted events, responses, code and source evidence an engineer can inspect. Then write checks for the equality or change the application needs. Expose a limitation when verification misses a class of drift, rather than broadening the meaning of a passing test. Keep external actions behind their own authorization and effect controls.

An event-sourced reactive graph can support proposals to change an agent's behavior, but selecting and validating those changes is a separate research and engineering problem. This account demonstrates neither a self-improving agent nor improved task outcomes. Its immediate value is a more precise way to reconstruct, inspect and compare the work that was actually captured, with enough versioned context to understand what remains unknown.

Acknowledgment and author information: review status

This preparation draft used OpenAI Codex (GPT-6.1 Sol) to generate and revise article text, figure source and verification code. Author review, responsibility, the complete history of any earlier AI assistance, funding/conflict statements, current affiliation, biography, ORCID and author photo remain to be confirmed in the separate worksheet. This review note must be resolved into the final truthful acknowledgments and author information before submission.

References

[1] Y. Nakajima, "The Log is the Agent: Event-Sourced Reactive Graphs for Auditable, Forkable Agentic Systems," arXiv:2605.21997v1, May 2026. `https://arxiv.org/abs/2605.21997v1`.

[2] ActiveGraph contributors, *ActiveGraph source*, revision 7070112 (full commit in the companion manifest), inspected Nov. 2, 2026. Pinned implementation. Companion evidence: the `verification.json` receipt in this article's source-owned evidence directory.

[3] M. Fowler, "Event Sourcing," Dec. 12, 2005. `https://martinfowler.com/eaaDev/EventSourcing.html`. Accessed Nov. 2, 2026.

[4] H. P. Nii, "PART ONE: The Blackboard Model of Problem Solving and the Evolution of Blackboard Architectures," *AI Magazine*, vol. 7, no. 2, pp. 38–53, 1986. doi: 10.1609/aimag.v7i2.537.

[5] Temporal Technologies, "Temporal Workflow Execution overview" and "Temporal Workflow Definition," documentation, accessed Nov. 2, 2026. `https://docs.temporal.io/workflow-execution`; `https://docs.temporal.io/workflow-definition`.

[6] Temporal Technologies, "Events and Event History," documentation, accessed Nov. 2, 2026. `https://docs.temporal.io/workflow-execution/event`.

[7] LangChain, "Use time-travel," LangGraph Python documentation, accessed Nov. 2, 2026. `https://docs.langchain.com/oss/python/langgraph/use-time-travel`.